

Introduzione agli Algoritmi Randomizzati

Prova scritta del 10 dicembre 2025

A.A. 2025/2026

Docente: Gianluca De Marco

Esercizio 1 (30 punti)

- a) Descrivere l'algoritmo *Randomized k-Select* in pseudocodice.
- b) Si consideri la lista non ordinata

[12, 5, 30, 7, 25, 18, 2, 40].

Vogliamo trovare il 4° **elemento più piccolo** tramite l'algoritmo *Randomized k-Select*.

Fissiamo in anticipo una sequenza di pivot casuali: 18, 7, 12, 25, 30. Ogni volta che l'algoritmo deve scegliere un pivot, usa il prossimo elemento di questa sequenza: al primo passo il pivot è 18, al secondo (se necessario) è 7, al terzo (se necessario) è 12, e così via. Se l'esecuzione dell'algoritmo termina prima di consumare tutti i pivot elencati, i rimanenti vengono semplicemente ignorati.

Eseguire l'algoritmo *Randomized k-Select* seguendo l'ordine di pivot specificato, mostrando le partizioni prodotte ad ogni passo fino alla determinazione del 4° elemento più piccolo.

- c) Sia $C(n)$ il numero di confronti eseguiti dal *Randomized k-Select* su n elementi. Indichiamo con $s_1 < s_2 < \dots < s_n$ gli elementi della lista ordinati in senso crescente, e con $A_i = \{\text{il pivot scelto è } s_i\}$.

Usando la definizione di valore atteso tramite condizionamento,

$$\mathbb{E}[C(n)] = \sum_{i=1}^n \mathbb{E}[C(n) \mid A_i] \Pr(A_i),$$

esprimere $\mathbb{E}[C(n)]$ ricorsivamente in funzione di $\mathbb{E}[C(k)]$ per valori $k < n$, mettendo in evidenza il contributo del partizionamento e il termine ricorsivo. Concludere indicando un bound asintotico per $\mathbb{E}[C(n)]$.

- a) **Pseudocodice (Randomized k -Select).** L'algoritmo cerca il k -esimo elemento più piccolo in un array A di lunghezza n .

Algorithm 1 RANDOMIZED- k -SELECT(A, k)

Require: Array A di $n \geq 1$ elementi (tutti distinti), intero $k \in \{1, \dots, n\}$

Ensure: Il k -esimo elemento più piccolo di A

```

1: if  $n = 1$  then
2:   return  $A[1]$ 
3: end if
4: scegli un pivot  $p$  uniformemente a caso tra gli elementi di  $A$ 
5: partiziona  $A$  in  $L = \{x \in A : x < p\}$ ,  $E = \{p\}$ ,  $G = \{x \in A : x > p\}$ 
6:  $i \leftarrow |L| + 1$ 
7: if  $k < i$  then
8:   return RANDOMIZED- $k$ -SELECT( $L, k$ )
9: else if  $k = i$  then
10:  return  $p$ 
11: else
12:  return RANDOMIZED- $k$ -SELECT( $G, k - i$ )
13: end if

```

- b) **Esecuzione sulla lista con pivot prefissati.** Vogliamo trovare il 4° elemento più piccolo di

$$A = [12, 5, 30, 7, 25, 18, 2, 40], \quad k = 4.$$

Passo 1: pivot $p = 18$. Partizione rispetto a 18:

$$L = [12, 5, 7, 2], \quad E = [18], \quad G = [30, 25, 40].$$

Poiché $|L| = 4$ e $k = 4 \leq |L|$, ricorriamo su L con lo stesso k .

Passo 2: (su L) pivot $p = 7$. Ora $A' = L = [12, 5, 7, 2]$ e $k = 4$. Partizione rispetto a 7:

$$L' = [5, 2], \quad E' = [7], \quad G' = [12].$$

Qui $|L'| = 2$ e $|L'| + 1 = 3$; poiché $k = 4 > 3$, ricorriamo su G' con

$$k' = k - |L'| - 1 = 4 - 2 - 1 = 1.$$

Passo 3: (su G') array di una sola chiave. $G' = [12]$ e $k' = 1$, quindi l'algoritmo termina e restituisce 12.

Conclusione: il 4° elemento più piccolo è 12. (I pivot successivi 12, 25, 30 vengono ignorati perché l'esecuzione è già terminata.)

- c) **Ricorrenza per $\mathbb{E}[C(n)]$ via condizionamento e bound asintotico.**

Sia $C(n)$ il numero di confronti eseguiti da RANDOMIZED- k -SELECT su n elementi. Durante una chiamata su n elementi il partizionamento richiede $n - 1$ confronti. Assumiamo che il pivot sia scelto uniformemente a caso tra gli n elementi, quindi $\Pr(A_i) = 1/n$ per ogni i .

Sia k l'ordine statistico cercato, e indichiamo con $s_1 < \dots < s_n$ gli elementi ordinati. Se si verifica l'evento A_i (pivot = s_i), allora:

- se $i = k$, l'algoritmo termina dopo il partizionamento;
- se $i > k$, ricorre sul sottoarray di sinistra di taglia $i - 1$ cercando ancora il k -esimo;
- se $i < k$, ricorre sul sottoarray di destra di taglia $n - i$ cercando il $(k - i)$ -esimo.

In tutti i casi si paga il costo del partizionamento, quindi:

$$\mathbb{E}[C(n) \mid A_i] = (n - 1) + \begin{cases} \mathbb{E}[C(i - 1)] & \text{se } i > k, \\ 0 & \text{se } i = k, \\ \mathbb{E}[C(n - i)] & \text{se } i < k. \end{cases}$$

Applicando il condizionamento:

$$\begin{aligned} \mathbb{E}[C(n)] &= \sum_{i=1}^n \mathbb{E}[C(n) \mid A_i] \Pr(A_i) \\ &= \frac{1}{n} \sum_{i=1}^n \mathbb{E}[C(n) \mid A_i] \\ &\leq (n - 1) + \frac{1}{n} \sum_{i=1}^n \mathbb{E}[C(\max\{i - 1, n - i\})]. \end{aligned}$$

Osserviamo che la quantità $\max\{i - 1, n - i\}$ cambia forma a seconda che i sia prima o dopo la metà.

$$\begin{aligned} \sum_{i=1}^n \mathbb{E}[C(\max\{i - 1, n - i\})] &= \sum_{i=1}^{\lfloor n/2 \rfloor} \mathbb{E}[C(n - i)] + \sum_{i=\lfloor n/2 \rfloor + 1}^n \mathbb{E}[C(i - 1)] \\ &= \sum_{i=\lceil n/2 \rceil}^{n-1} \mathbb{E}[C(i)] + \sum_{i=\lceil n/2 \rceil}^{n-1} \mathbb{E}[C(i)] \\ &\leq 2 \sum_{i=\lceil n/2 \rceil}^{n-1} \mathbb{E}[C(i)]. \end{aligned}$$

Quindi

$$\mathbb{E}[C(n)] \leq (n - 1) + \frac{2}{n} \sum_{t=\lceil n/2 \rceil}^{n-1} \mathbb{E}[C(t)]$$

Da questa ricorrenza segue (come visto a lezione) che

$$\boxed{\mathbb{E}[C(n)] = O(n)}.$$

Più precisamente, è noto che esiste una costante c tale che $\mathbb{E}[C(n)] \leq cn$ per ogni n (ad esempio si può mostrare $\mathbb{E}[C(n)] \leq 4n$ con un'analisi standard).

Esercizio 2 (25 punti)

- a) Descrivere l'algoritmo di *Freivalds* in pseudocodice e mostrare che esso usa soltanto $O(n^2)$ operazioni.
- b) Si considerino le matrici

$$A = \begin{pmatrix} 1 & 2 \\ 0 & 1 \end{pmatrix}, \quad B = \begin{pmatrix} 1 & 1 \\ 1 & 0 \end{pmatrix}, \quad C = \begin{pmatrix} 3 & 1 \\ 1 & 0 \end{pmatrix}.$$

Vogliamo usare l'algoritmo di Freivalds per verificare se $AB = C$. Supponiamo che il vettore casuale usato dall'algoritmo sia il seguente:

$$r = \begin{pmatrix} 1 \\ 0 \end{pmatrix}.$$

Stabilire se, in *questa* esecuzione dell'algoritmo di Freivalds, la verifica $AB = C$ ha esito positivo oppure negativo.

- c) Spiegare chiaramente il tipo di errore che l'algoritmo di Freivalds può commettere e come la probabilità di tale errore possa essere resa molto piccola.

a) **Algoritmo di Freivalds e complessità.**

Algorithm 2 FREIVALDS(A, B, C)

Require: Matrici $A, B, C \in \mathbb{R}^{n \times n}$

Ensure: ACCEPT o REJECT

```

1: scegli  $r \in \{0, 1\}^n$  uniformemente a caso
2:  $x \leftarrow Br$   $\triangleright O(n^2)$ 
3:  $y \leftarrow Ax$   $\triangleright O(n^2)$ 
4:  $z \leftarrow Cr$   $\triangleright O(n^2)$ 
5: if  $y = z$  then
6:   return ACCEPT
7: else
8:   return REJECT
9: end if

```

Costo. Ogni prodotto matrice-vettore costa $O(n^2)$ operazioni aritmetiche. Si calcolano tre prodotti matrice-vettore (Br , $A(Br)$, Cr), quindi il costo totale è

$$O(n^2) + O(n^2) + O(n^2) = O(n^2).$$

(Confrontare due vettori costa $O(n)$, trascurabile rispetto a $O(n^2)$.)

b) **Esecuzione con $r = (1, 0)^T$.**

Calcoliamo i tre vettori.

1) $x = Br$:

$$Br = \begin{pmatrix} 1 & 1 \\ 1 & 0 \end{pmatrix} \begin{pmatrix} 1 \\ 0 \end{pmatrix} = \begin{pmatrix} 1 \\ 1 \end{pmatrix}.$$

2) $y = A(Br) = Ax$:

$$Ax = \begin{pmatrix} 1 & 2 \\ 0 & 1 \end{pmatrix} \begin{pmatrix} 1 \\ 1 \end{pmatrix} = \begin{pmatrix} 1+2 \\ 1 \end{pmatrix} = \begin{pmatrix} 3 \\ 1 \end{pmatrix}.$$

3) $z = Cr$:

$$Cr = \begin{pmatrix} 3 & 1 \\ 1 & 0 \end{pmatrix} \begin{pmatrix} 1 \\ 0 \end{pmatrix} = \begin{pmatrix} 3 \\ 1 \end{pmatrix}.$$

Poiché $y = z$, in questa esecuzione l'algoritmo restituisce **ACCEPT**, cioè la verifica $AB = C$ ha **esito positivo**.

(Nota: questo non implica che $AB = C$ sia vero; significa solo che il test non ha trovato evidenza di errore con questo r .)

c) **Tipo di errore e come ridurlo.**

L'algoritmo di Freivalds è un algoritmo di tipo *Monte Carlo a errore unilaterale*:

- se $AB = C$, allora $A(Br) = Cr$ per ogni r , quindi l'algoritmo accetta sempre (nessun falso negativo);
- se $AB \neq C$, allora l'algoritmo può comunque accettare per alcuni r (possibile *falso positivo*).

Più precisamente, ponendo $D = AB - C$, l'algoritmo accetta quando $Dr = 0$. Scegliendo $r \in \{0, 1\}^n$ uniforme, si può mostrare che

$$\Pr[Dr = 0] \leq \frac{1}{2}.$$

Quindi una singola esecuzione sbaglia con probabilità al più $1/2$ nel caso $AB \neq C$.

Per rendere l'errore molto piccolo, si ripete l'algoritmo t volte con vettori r scelti in modo indipendente e si accetta solo se tutte le prove accettano. Allora la probabilità di errore diventa al più

$$\left(\frac{1}{2}\right)^t.$$

Esercizio 3 (25 punti)

Consideriamo il modello *balls and bins* in cui n palline vengono lanciate in modo indipendente e uniforme in n urne. Per un'urna fissata $j \in \{1, \dots, n\}$, indichiamo con B_j il numero di palline che finiscono in j .

- a) Enunciare il teorema sul *carico massimo* (massimo numero di palline che finiscono in una singola urna), così come è stato studiato a lezione.
- b) Determinare la distribuzione di B_j e il valore atteso $\mathbb{E}[B_j]$, fornendo tutti i passaggi (inclusa la definizione di variabili indicatrici). Indicare inoltre se $B_1, B_2, \dots, B_n$ sono variabili aleatorie dipendenti o indipendenti motivando la risposta.
- c) Indicare in quale punto della dimostrazione del teorema del carico massimo viene applicato il *Chernoff bound* e rendere esplicita l'applicazione scrivendo chiaramente i parametri μ e ε coinvolti.

- a) **Teorema sul carico massimo.** Nel modello *balls and bins* con n palline lanciate in modo indipendente e uniforme in n urne, sia B_j il carico dell'urna j e sia

$$M = \max_{1 \leq j \leq n} B_j$$

il carico massimo. Allora, per n sufficientemente grande, con probabilità almeno $1 - \frac{1}{n}$ vale

$$M \leq k \quad \text{dove} \quad k = \frac{3 \ln n}{\ln \ln n} - 1.$$

Equivalentemente,

$$\Pr[M > k] < \frac{1}{n}.$$

(In particolare, con alta probabilità $M = O(\frac{\ln n}{\ln \ln n})$.)

- b) **Distribuzione di B_j , valore atteso, dipendenza.**

Per ogni pallina $\ell \in \{1, \dots, n\}$ definiamo la variabile indicatrice

$$X_\ell = \begin{cases} 1 & \text{se la pallina } \ell \text{ finisce nell'urna } j, \\ 0 & \text{altrimenti.} \end{cases}$$

Allora

$$B_j = \sum_{\ell=1}^n X_\ell.$$

Poiché ogni pallina finisce in j con probabilità $1/n$ e i lanci sono indipendenti, le variabili $X_1, \dots, X_n$ sono i.i.d. con distribuzione di Bernoulli($1/n$). Di conseguenza B_j ha distribuzione binomiale:

$$B_j \sim \text{Bin}\left(n, \frac{1}{n}\right), \quad \text{cioè} \quad \Pr[B_j = t] = \binom{n}{t} \left(\frac{1}{n}\right)^t \left(1 - \frac{1}{n}\right)^{n-t}, \quad t = 0, 1, \dots, n.$$

Per il valore atteso:

$$\mathbb{E}[B_j] = \mathbb{E}\left[\sum_{\ell=1}^n X_\ell\right] = \sum_{\ell=1}^n \mathbb{E}[X_\ell] = \sum_{\ell=1}^n \Pr[X_\ell = 1] = n \cdot \frac{1}{n} = 1.$$

Dipendenza/indipendenza di $B_1, \dots, B_n$. Le variabili $B_1, \dots, B_n$ *non sono indipendenti*. Infatti soddisfano sempre la relazione deterministica

$$B_1 + B_2 + \dots + B_n = n,$$

il che esclude l'indipendenza. Ad esempio, l'evento $\{B_1 = n\}$ implica $\{B_2 = 0\}$, quindi $\Pr[B_2 = 0 \mid B_1 = n] = 1 \neq \Pr[B_2 = 0]$, essendo $\Pr[B_2 = 0] = \left(1 - \frac{1}{n}\right)^n < 1$.

- c) **Dove si usa Chernoff e applicazione con μ e ε .**

Nella dimostrazione del teorema sul carico massimo si considera l'evento

$$\{M > k\} = \left\{ \max_{1 \leq j \leq n} B_j > k \right\}$$

e si applica il metodo dell'unione (*union bound*):

$$\Pr[M > k] = \Pr\left[\bigcup_{j=1}^n \{B_j > k\}\right] \leq \sum_{j=1}^n \Pr[B_j > k] = n \Pr[B_1 > k],$$

dove l'ultima uguaglianza usa il fatto che le urne sono identicamente distribuite (quindi $\Pr[B_j > k]$ è la stessa per ogni j).

Il *Chernoff bound* viene applicato per stimare $\Pr[B_1 > k]$ (cioè la probabilità che una singola urna abbia più di k palline). Poiché $B_1 \sim \text{Bin}(n, 1/n)$, abbiamo

$$\mu = \mathbb{E}[B_1] = 1.$$

Scriviamo l'evento $\{B_1 > k\}$ nella forma standard di Chernoff:

$$B_1 > k \iff B_1 \geq k+1 \iff B_1 \geq (1+\varepsilon)\mu,$$

dove, poiché $\mu = \mathbb{E}[B_1] = 1$, scegliamo $\varepsilon = k$.

Applicando il bound di Chernoff, otteniamo, con $\mu = 1$ e $\varepsilon = k$,

$$\Pr[B_1 > k] = \Pr[B_1 \geq (1+\varepsilon)\mu] < \left(\frac{e^k}{(1+k)^{1+k}}\right)^\mu = \frac{e^k}{(1+k)^{1+k}}.$$

A questo punto combiniamo tale stima con l'union bound:

$$\Pr[M > k] \leq n \Pr[B_1 > k] < n \cdot \frac{e^k}{(k+1)^{k+1}}.$$

Scegliendo

$$k = \frac{3 \ln n}{\ln \ln n} - 1,$$

si può dimostrare che, per n sufficientemente grande,

$$\Pr[M > k] \leq \frac{1}{n},$$

che è esattamente la tesi del teorema.

Esercizio 4 (20 punti)

Nel modello di hashing con *random probing* (open addressing con sequenze di probing uniformi), si assume che ad ogni tentativo di inserimento o ricerca, la posizione successiva nella tabella venga scelta uniformemente a caso tra le posizioni libere.

- a) Sia X il costo di una ricerca non riuscita. Spiegare perché

$$\Pr[X > i] = \frac{m}{n} \cdot \frac{m-1}{n-1} \cdot \frac{m-2}{n-2} \cdots \frac{m-i+1}{n-i+1} \leq \alpha^i,$$

dove m è il numero di chiavi memorizzate nella tabella, n è la dimensione della tabella e $\alpha = m/n$ è il fattore di carico.

- b) Nella dimostrazione che il costo atteso di una ricerca non riuscita è al più $1/(1-\alpha)$, è essenziale che $|\alpha| < 1$. Spiegare brevemente quale passaggio dell'analisi non sarebbe più lecito se $\alpha \geq 1$.

- a) Una ricerca non riuscita continua finché le celle visitate risultano tutte occupate. Dunque l'evento $\{X > i\}$ coincide con l'evento che i primi i tentativi cadano tutti su celle occupate. Definiamo, per ogni $t \geq 1$, l'evento $A_t = \{\text{la } t\text{-esima cella esaminata è occupata}\}$. Per definizione del costo X di una ricerca non riuscita, l'evento $\{X > i\}$ equivale al fatto che le prime i celle esaminate siano tutte occupate, cioè

$$\{X > i\} \iff A_1 \cap A_2 \cap \dots \cap A_i.$$

In particolare,

$$\Pr[X > 0] = 1, \quad \Pr[X > 1] = \Pr[A_1], \quad \Pr[X > 2] = \Pr[A_1 \cap A_2], \dots$$

e in generale, usando la regola della catena,

$$\Pr[X > i] = \Pr[A_1 \cap A_2 \cap \dots \cap A_i] = \Pr[A_1] \cdot \Pr[A_2 \mid A_1] \cdots \Pr[A_i \mid A_1 \cap \dots \cap A_{i-1}].$$

Condizionatamente a $A_1 \cap \dots \cap A_{t-1}$ abbiamo già visto $(t-1)$ celle occupate, quindi restano $n - (t-1)$ celle non ancora provate, di cui $m - (t-1)$ occupate. Pertanto

$$\Pr[A_t \mid A_1 \cap \dots \cap A_{t-1}] = \frac{m - (t-1)}{n - (t-1)}.$$

Sostituendo nella catena otteniamo, per $1 \leq i \leq m$,

$$\Pr[X > i] = \prod_{t=1}^i \frac{m - (t-1)}{n - (t-1)} = \frac{m}{n} \cdot \frac{m-1}{n-1} \cdot \frac{m-2}{n-2} \cdots \frac{m-i+1}{n-i+1}.$$

Chiaramente per $i > m$ si ha $\Pr[X > i] = 0$. Quindi

$$\Pr[X > i] = \prod_{t=0}^{i-1} \frac{m-t}{n-t} \leq \prod_{t=0}^{i-1} \frac{m}{n} = \alpha^i.$$

- b) Per stimare il costo atteso di una ricerca non riuscita si usa la formula di coda

$$\mathbb{E}[X] = \sum_{i \geq 0} \Pr[X > i]$$

e poi il bound del punto precedente:

$$\mathbb{E}[X] \leq \sum_{i \geq 0} \alpha^i.$$

Il passaggio successivo,

$$\sum_{i \geq 0} \alpha^i = \frac{1}{1 - \alpha},$$

è lecito soltanto se $|\alpha| < 1$ (qui $\alpha \geq 0$, quindi serve $\alpha < 1$), perché è la somma di una serie geometrica convergente. Se $\alpha \geq 1$ la serie geometrica diverge e tale bound non è applicabile; in particolare, per $\alpha = 1$ non esistono celle libere e una ricerca non riuscita (non avendo una cella vuota da incontrare) non termina nel nostro modello.